

Brief 1 — Fine-tuning initial et évaluation

CONTEXTE DU PROJET

Votre aventure chez **FastIA** continue. L'entreprise reçoit quotidiennement des centaines de demandes clients en texte libre : tickets support, demandes commerciales, réclamations, projets de transformation. Aujourd'hui, ces demandes sont traitées manuellement, ce qui génère des délais et des erreurs de routage.

Votre manager vous confie une première mission : **adapter un modèle de langage existant pour qu'il apprenne à classifier automatiquement ces demandes et générer une réponse structurée**. C'est votre premier pas dans le cycle de développement d'une solution IA : comprendre les données, entraîner un modèle, mesurer ce qu'il vaut.

OBJECTIF PRINCIPAL

Fine-tuner **Llama 3.2 1B** sur le dataset FastIA, évaluer les résultats et produire un diagnostic argumenté sur la qualité du modèle obtenu.

DATASET

Un dataset de **100 exemples** vous est fourni : [dataset_fastia_module1.json](#)

Chaque exemple suit cette structure :

```
{
  "input": "texte de la demande client",
  "output": {
    "categorie": "...",
    "priorite": "...",
    "reponse_suggeree": "..."
  }
}
```

Catégories possibles :

[Support technique](#) [Demande commerciale](#) [Demande de transformation](#) [Réclamation](#)
[Information générale](#)

Niveaux de priorité : [haute](#) [normale](#)

ÉTAPES DU PROJET

1 Mise en place de l'environnement

- Créer un environnement virtuel Python
- Installer les dépendances :

```
pip install transformers peft datasets torch mlflow accelerate bitsandbytes
```

- Initialiser un dépôt GitHub et versionner dès le départ

2 Exploration du dataset

⚠ Cette étape est obligatoire. Un modèle ne peut pas être meilleur que ses données.

- Quelle est la distribution des catégories ?
- Quelle est la distribution des priorités ?
- Le dataset est-il équilibré ?
- Quelle est la longueur moyenne des demandes en entrée ?

3 Préparation du dataset

- Construire un **prompt template** d'instruction pour chaque exemple :

```
<s>[INST] Tu es un assistant FastIA. Analyse la demande suivante et réponds uniquement en JSON valide avec les champs : categorie, priorite, reponse_suggeree.
```

```
Demande : {input} [/INST]
{output_json} </s>
```

- Diviser le dataset : **80% entraînement / 20% test**
- Vérifier que la distribution des catégories est représentée dans les deux jeux

4 Fine-tuning avec LoRA

- Charger le modèle de base [meta-llama/Llama-3.2-1B](#) depuis HuggingFace Hub
- Configurer LoRA avec PEFT :

Paramètre LoRA	Valeur
<code>r</code>	8
<code>lora_alpha</code>	16
<code>target_modules</code>	<code>["q_proj", "v_proj"]</code>

- Lancer **un run d'entraînement** avec les paramètres :

Paramètre	Valeur
learning_rate	2e-4
num_train_epochs	3
per_device_train_batch_size	4

- Tracker le run dans **MLflow** :
 - Paramètres : learning rate, batch size, nombre d'époques, configuration LoRA
 - Métriques : loss d'entraînement, loss de validation par époque

5 Évaluation quantitative

- Analyser la courbe de loss : descend-elle régulièrement ? Se stabilise-t-elle ?
- Comparer train loss et eval loss : y a-t-il un écart important ?
- Exporter le rapport MLflow

6 Évaluation qualitative

- Tester le modèle sur **10 demandes de votre choix**, couvrant les 5 catégories
- Pour chaque prédiction, vérifier :
 - Le JSON généré est-il valide ?
 - La catégorie est-elle correcte ?
 - La priorité est-elle cohérente avec la demande ?
 - La réponse suggérée est-elle pertinente ?

7 Diagnostic

Sur la base des évaluations quantitative et qualitative, produire un **diagnostic écrit** répondant aux questions suivantes :

- Le modèle classe-t-il correctement toutes les catégories ou certaines posent-elles problème ?
- Les erreurs sont-elles aléatoires ou systématiques ?
- La loss indique-t-elle un sous-apprentissage ou un sur-apprentissage ?
- Quelles sont selon vous les causes probables des erreurs observées ?
- Quelles actions correctives proposez-vous : compléter le dataset, ajuster les hyperparamètres, modifier le prompt template ?



Ce diagnostic sera le point de départ du Brief 2.

LIVRABLES

Dépôt GitHub versionné

- Le notebook de fine-tuning et d'évaluation
- Le dataset utilisé

README.md

- Le cas d'usage et l'objectif
- L'architecture du projet
- Les étapes d'installation et de lancement
- Les résultats de l'entraînement
- Le diagnostic et les actions correctives envisagées

Rapport MLflow exporté

- Captures du run avec courbes de loss et métriques

CHARGE DE TRAVAIL ESTIMÉE

 4 à 6 heures

RESSOURCES

- [HuggingFace Hub — Llama 3.2 1B](#)
- [Documentation PEFT](#)
- [Documentation MLflow](#)

BONUS

- Calculer la précision par catégorie sur le jeu de test
- Produire une matrice de confusion pour visualiser les erreurs de classification